\# Matriz de Trazabilidad Integral

\## VotaGT - Sistema de Gestión Electoral

Versión: 1.0

Fecha: Octubre 2026

\---

\# 1. Objetivo

La presente matriz relaciona los diferentes artefactos desarrollados durante el proyecto VotaGT.

La trazabilidad permite demostrar la correspondencia entre:

\- Requisitos.

\- Historias de Usuario.

\- Arquitectura.

\- Contrato OpenAPI.

\- Código fuente.

\- Escenarios BDD.

\- Casos de prueba.

\- Evidencias.

\---

\# 2. Trazabilidad de requisitos funcionales

| Requisito | HU | Funcionalidad | C4 | Endpoint OpenAPI | Código Backend | BDD | Prueba / Evidencia | Estado |

```
|---|---|---|---|---|---|---|---|---|

| REQ-FN-001 | HU-01 | Autenticación | N2/N3 | POST `/auth/login` |
`auth.controller.js`, `auth.service.js`, `auth.repository.js` |
`autenticacion.feature` | CP-01 Login correcto / CP-02 Login incorrecto |
Implementado |

| REQ-FN-002 | HU-01 | Consultar perfil | N2/N3 | GET `/auth/perfil` |
`auth.controller.js`, `auth.service.js`, `auth.middleware.js` |
`autenticacion.feature` | Postman Perfil + `api.test.js` | Implementado |

| REQ-FN-003 | HU-05 | Listar votaciones | N2/N3 | GET `/votaciones` |
`votacion.controller.js`, `votacion.service.js`, `votacion.repository.js` |
`gestion\_electoral.feature` | Postman + `api.test.js` | Implementado |

| REQ-FN-004 | HU-05 | Crear votación | N2/N3 | POST `/votaciones` |
`votacion.controller.js`, `votacion.service.js`, `votacion.repository.js` |
`gestion\_electoral.feature` | CP-04 Votación | Implementado |

| REQ-FN-005 | HU-05 | Consultar votación por ID | N2/N3 | GET
`/votaciones/{votacionId}` | `votacion.controller.js`, `votacion.service.js` |
`gestion\_electoral.feature` | Postman + `api.test.js` | Implementado |

| REQ-FN-006 | HU-07 | Consultar candidatos | N2/N3 | GET
`/votaciones/{votacionId}/candidatos` | `candidato.controller.js`,
`candidato.service.js`, `candidato.repository.js` | `gestion\_electoral.feature` |
Postman | Implementado |

| REQ-FN-007 | HU-07 | Registrar candidato | N2/N3 | POST
`/votaciones/{votacionId}/candidatos` | `candidato.controller.js`,
`candidato.service.js`, `candidato.repository.js` | `gestion\_electoral.feature` |
CP-06 Candidato | Implementado |

| REQ-FN-008 | HU-06 | Consultar mesas | N2/N3 | GET
`/votaciones/{votacionId}/mesas` | `mesa.controller.js`, `mesa.service.js`,
`mesa.repository.js` | `gestion\_electoral.feature` | Postman | Implementado |

| REQ-FN-009 | HU-06 | Registrar mesa | N2/N3 | POST
`/votaciones/{votacionId}/mesas` | `mesa.controller.js`, `mesa.service.js`,
`mesa.repository.js` | `gestion\_electoral.feature` | CP-05 Mesa | Implementado |

| REQ-FN-010 | HU-08 | Registrar resultado | N2/N3 | POST
`/votaciones/{votacionId}/resultados` | `resultado.controller.js`,
`resultado.service.js`, `resultado.repository.js` | `gestion\_electoral.feature` |
CP-07 Resultado | Implementado |

| REQ-FN-011 | HU-09 | Consultar resultados | N2/N3 | GET
`/votaciones/{votacionId}/resultados` | `resultado.service.js`,
`resultado.repository.js`, `cache.gateway.js` | `gestion\_electoral.feature` |
CP-08 Resultados + HIT/MISS | Implementado |

| REQ-FN-012 | HU-11 | Auditoría de eventos | N2/N3 | Operación interna |
`eventBus.js`, `audit.observer.js`, `audit.service.js` |
`gestion\_electoral.feature` | Evidencia MongoDB | Implementado |

| REQ-FN-013 | HU-10 | Exportar resultados | N2/N3 | GET
`/votaciones/{votacionId}/resultados/exportar` | `resultado.service.js`,
`resultado.controller.js`, `resultados.strategy.js` | `gestion\_electoral.feature`
```

| CP-09 CSV + Postman JSON/formato inválido | Implementado |

\---

\# 3. Historias pendientes del backlog

Las siguientes Historias de Usuario se mantienen en el Product Backlog, pero no se consideran implementadas actualmente.

Historia	Funcionalidad	Estado	Observación
HU-02	Crear usuarios	Pendiente	No existe endpoint funcional terminado
HU-03	Editar usuarios	Pendiente	No existe endpoint funcional terminado
HU-04	Administrar roles y permisos	Pendiente	La autorización existe, pero no se implementó CRUD administrativo
HU-12	Documentación técnica	En progreso	Se completa durante la entrega final

Esto evita presentar como terminadas funcionalidades que aún no cuentan con implementación y evidencia.

\---

\# 4. Trazabilidad de requisitos no funcionales

Requisito	Atributo	Mecanismo	Código / Artefacto	Evidencia	Estado
REQ-NF-001	Seguridad	JWT	`auth.middleware.js`	401 sin token	Implementado
REQ-NF-002	Autorización	Permisos	`permission.middleware.js`	403 sin permiso + pruebas unitarias	Implementado
REQ-NF-003	Manejo de errores	RFC 9457	`errorHandler.js`	`application/problem+json`	Implementado

| REQ-NF-004 | Rendimiento | Redis Cache-Aside | `cache.gateway.js`, `resultado.service.js` | CACHE MISS / HIT | Implementado |

| REQ-NF-005 | Disponibilidad | Fallback PostgreSQL | `cache.gateway.js` | Redis DEGRADED + HTTP 200 | Implementado |

| REQ-NF-006 | Resiliencia | Circuit Breaker | `circuitBreaker.js` | OPEN / HALF_OPEN / CLOSED | Implementado |

| REQ-NF-007 | Integridad | Restricciones PostgreSQL | `database\_complete.sql` | Restauración `votagt\_test` | Implementado |

| REQ-NF-008 | Auditabilidad | Observer + MongoDB | `eventBus.js`, `audit.observer.js` | Eventos MongoDB | Implementado |

| REQ-NF-009 | Mantenibilidad | Arquitectura modular | `src/` | SDD + UML Paquetes | Implementado |

| REQ-NF-010 | Interoperabilidad | REST + OpenAPI 3.0.3 | `openapi.yaml` | Swagger Editor | Implementado |

| REQ-NF-011 | Configurabilidad | Variables de entorno | `.env.example` | Configuración local | Implementado |

| REQ-NF-012 | Capacidad de prueba | node:test / Postman / BDD | `tests/`, `features/` | `npm test` con `fail 0` | Implementado |

\---

\# 5. Trazabilidad de patrones de diseño

Patrón	Requisito relacionado	Implementación	Evidencia
Repository	REQ-NF-009	`src/repositories/`	Acceso PostgreSQL desacoplado
Gateway	REQ-NF-004	`cache.gateway.js`	Redis HIT/MISS
Observer	REQ-FN-012	`eventBus.js`, `audit.observer.js`	Eventos almacenados en MongoDB
Circuit Breaker	REQ-NF-005 / REQ-NF-006	`circuitBreaker.js`	Redis DEGRADED / OPEN / HALF_OPEN / CLOSED
Strategy	REQ-FN-013	`resultados.strategy.js`	Exportación JSON y CSV

\---

\# 6. Trazabilidad de persistencia políglota

Tecnología	Requisito	Responsabilidad	Evidencia
PostgreSQL	REQ-NF-007	Datos transaccionales	DDL + restauración
Redis	REQ-NF-004	Caché	HIT / MISS / TTL
MongoDB	REQ-FN-012	Auditoría	Colección `auditoria`
Neo4j	REQ-NF-002	Grafo de autorización	Usuario -> Rol -> Permiso

\---

\# 7. Trazabilidad BDD

\## Autenticación

Archivo:

`features/autenticacion.feature`

Relaciona principalmente:

\- REQ-FN-001.
\- REQ-FN-002.
\- REQ-NF-001.
\- REQ-NF-002.
\- REQ-NF-003.

\---

\## Gestión electoral

Archivo:

`features/gestion\_electoral.feature`

Relaciona:

\- REQ-FN-003.

\- REQ-FN-004.

\- REQ-FN-005.

\- REQ-FN-006.

\- REQ-FN-007.

\- REQ-FN-008.

\- REQ-FN-009.

\- REQ-FN-010.

\- REQ-FN-011.

\- REQ-FN-012.

\- REQ-FN-013.

También incluye escenarios relacionados con:

\- Errores 400.

\- Recursos 404.

\- Conflictos 409.

\- Cache HIT.

\- Cache MISS.

\- Exportación CSV.

\- Exportación JSON.

\- Formato inválido.

\---

\# 8. Trazabilidad con pruebas automatizadas

\## Tests unitarios

Ubicación:

`tests/unit/`

Cobertura actual:

\- Permission Middleware.
\- Strategy.
\- Circuit Breaker.

\## Tests de integración

Ubicación:

`tests/integration/`

Cobertura:

\- Health.
\- Login.
\- Perfil.
\- Autenticación requerida.
\- Consulta de votaciones.

\- Recurso inexistente.

Criterio general:

`npm test`

deberá finalizar con:

`fail 0`

\---

\# 9. Trazabilidad con Postman

Colección:

`docs/api/postman\_collection.json`

Contiene pruebas para:

\- Autenticación.
\- Perfil.
\- Votaciones.
\- Candidatos.
\- Mesas.
\- Resultados.
\- Cache MISS.
\- Cache HIT.
\- Exportación CSV.

\- Exportación JSON.

\- Formato inválido.

\- Casos negativos.

\---

\# 10. Trazabilidad documental

Artefacto	Ubicación
SRS ISO 29148	`docs/requirements/srs\_ieee\_29148.md`
SDD IEEE 1016	`docs/requirements/fichas\_diseno\_ieee1016.md`
UML	`docs/uml/`
Backlog	`docs/agile/backlog.md`
ADR	`docs/architecture/`
ISO 25010	`docs/architecture/escenarios\_iso25010.md`
OpenAPI	`docs/api/openapi.yaml`
Postman	`docs/api/postman\_collection.json`
Persistencia	`docs/database/`
UX/UI	`docs/ui/reporte\_ux\_ui.md`
SQA	`docs/qa/plan\_sqa\_ieee730.md`
Defectos	`docs/qa/reporte\_defectos\_ieee1044.md`
BDD	`features/`
Tests	`tests/`

\---

\# 11. Evaluación de cobertura

La matriz permite comprobar que las funcionalidades implementadas poseen relación

con al menos uno de los siguientes elementos:

\- Requisito.

\- Historia de Usuario.

\- Endpoint.

\- Componente backend.

\- Escenario BDD.

\- Prueba.

\- Evidencia.

Las historias todavía pendientes se identifican explícitamente para evitar una falsa declaración de cobertura.

\---

\# 12. Conclusión

La trazabilidad integral de VotaGT demuestra la relación entre análisis, diseño, implementación y pruebas.

Los requisitos implementados pueden seguirse desde la especificación SRS hasta el código fuente y las evidencias de prueba, proporcionando una estructura verificable para la evaluación final del proyecto.